

In previous chapters we considered combinational circuits where the value of each output depends solely on the values of signals applied to the inputs. There exists another class of logic circuits in which the values of the outputs depend not only on the present values of the inputs but also on the past behavior of the circuit. Such circuits include storage elements that store the values of logic signals. The contents of the storage elements are said to represent the *state* of the circuit. When the circuit's inputs change values, the new input values either leave the circuit in the same state or cause it to change into a new state. Over time the circuit changes through a sequence of states as a result of changes in the inputs. Circuits that behave in this way are referred to as *sequential circuits*.

In this chapter we will introduce circuits that can be used as storage elements. But first, we will motivate the need for such circuits by means of a simple example. Suppose that we wish to control an alarm system, as shown in Figure 7.1. The alarm mechanism responds to the control input *On/Off*. It is turned on when $\text{On}/\overline{\text{Off}} = 1$, and it is off when $\text{On}/\overline{\text{Off}} = 0$. The desired operation is that the alarm turns on when the sensor generates a positive voltage signal, *Set*, in response to some undesirable event. Once the alarm is triggered, it must remain active even if the sensor output goes back to zero. The alarm is turned off manually by means of a *Reset* input. The circuit requires a memory element to remember that the alarm has to be active until the *Reset* signal arrives.

Figure 7.2 gives a rudimentary memory element, consisting of a loop that has two inverters. If we assume that $A = 0$, then $B = 1$. The circuit will maintain these values indefinitely. We say that the circuit is in the *state* defined by these values. If we assume that $A = 1$, then $B = 0$, and the circuit will remain in this second state indefinitely. Thus the circuit has two possible states. This circuit is not useful, because it lacks some practical means for changing its state.

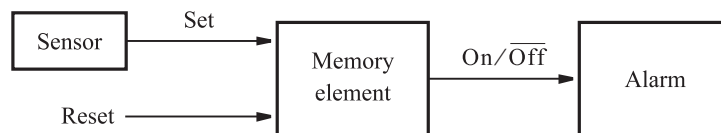


Figure 7.1 Control of an alarm system.

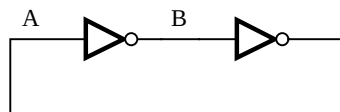


Figure 7.2 A simple memory element.

7.1 BASIC LATCH

Instead of using the transmission gates, we can construct a similar circuit using ordinary logic gates. Figure 7.4 presents a memory element built with NOR gates. Its inputs, *Set* and *Reset*, provide the means for changing the state, Q , of the circuit. A more usual way of drawing this circuit is given in Figure 7.5a, where the two NOR gates are said to be connected in cross-coupled style. The circuit is referred to as a *basic latch*. Its behavior is described by the table in Figure 7.5b. When both inputs, R and S , are equal to 0 the latch maintains its existing state. This state may be either $Q_a = 0$ and $Q_b = 1$, or $Q_a = 1$ and $Q_b = 0$, which is indicated in the table by stating that the Q_a and Q_b outputs have values

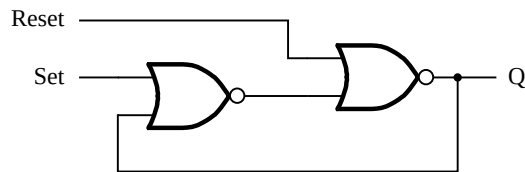


Figure 7.4 A memory element with NOR gates.

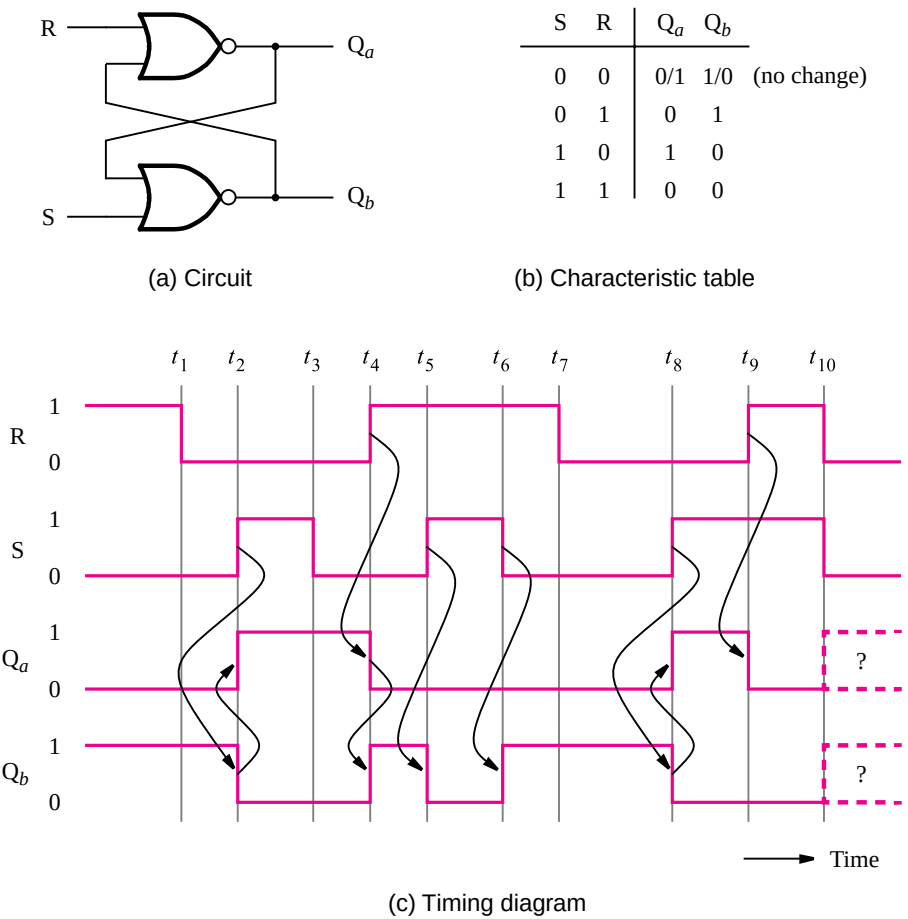


Figure 7.5 A basic latch built with NOR gates.

0/1 and 1/0, respectively. Observe that Q_a and Q_b are complements of each other in this case. When $R = 0$ and $S = 1$, the latch is *set* into a state where $Q_a = 1$ and $Q_b = 0$. When $R = 1$ and $S = 0$, the latch is *reset* into a state where $Q_a = 0$ and $Q_b = 1$. The fourth possibility is to have $R = S = 1$. In this case both Q_a and Q_b will be 0. The table in Figure 7.5b resembles a truth table. However, since it does not represent a combinational circuit in which the values of the outputs are determined solely by the current values of the inputs, it is often called a *characteristic table* rather than a truth table.

Figure 7.5c gives a timing diagram for the latch, assuming that the propagation delay through the NOR gates is negligible. Of course, in a real circuit the changes in the waveforms would be delayed according to the propagation delays of the gates. We assume that initially $Q_a = 0$ and $Q_b = 1$. The state of the latch remains unchanged until time t_2 , when S becomes equal to 1, causing Q_b to change to 0, which in turn causes Q_a to change to 1.

The causality relationship is indicated by the arrows in the diagram. When S goes to 0 at t_3 , there is no change in the state because both S and R are then equal to 0. At t_4 we have $R = 1$, which causes Q_a to go to 0, which in turn causes Q_b to go to 1. At t_5 both S and R are equal to 1, which forces both Q_a and Q_b to be equal to 0. As soon as S returns to 0, at t_6 , Q_b becomes equal to 1 again. At t_8 we have $S = 1$ and $R = 0$, which causes $Q_b = 0$ and $Q_a = 1$. An interesting situation occurs at t_{10} . From t_9 to t_{10} we have $Q_a = Q_b = 0$ because $R = S = 1$. Now if both R and S change to 0 at t_{10} , both Q_a and Q_b will go to 1. But having both Q_a and Q_b equal to 1 will immediately force $Q_a = Q_b = 0$. There will be an oscillation between $Q_a = Q_b = 0$ and $Q_a = Q_b = 1$. If the delays through the two NOR gates are exactly the same, the oscillation will continue indefinitely. In a real circuit there will invariably be some difference in the delays through these gates, and the latch will eventually settle into one of its two stable states, but we don't know which state it will be. This uncertainty is indicated in the waveforms by dashed lines.

The oscillations discussed above illustrate that even though the basic latch is a simple circuit, careful analysis has to be done to fully appreciate its behavior. In general, any circuit that contains one or more feedback paths, such that the state of the circuit depends on the propagation delays through logic gates, has to be designed carefully. We discuss timing issues in detail in Chapter 9.

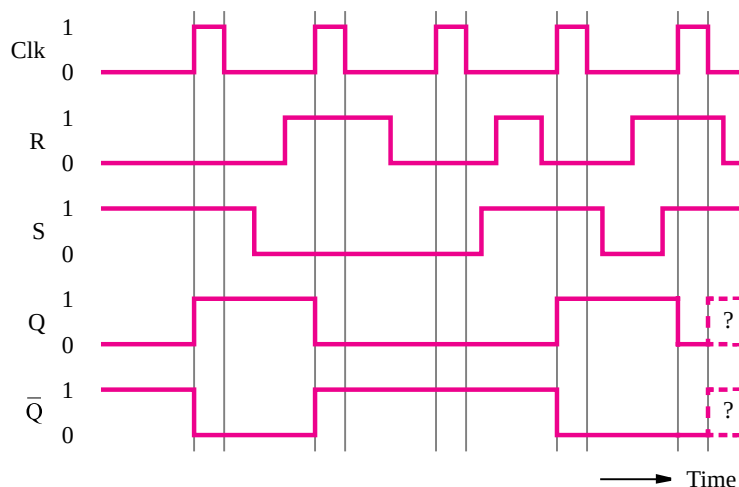
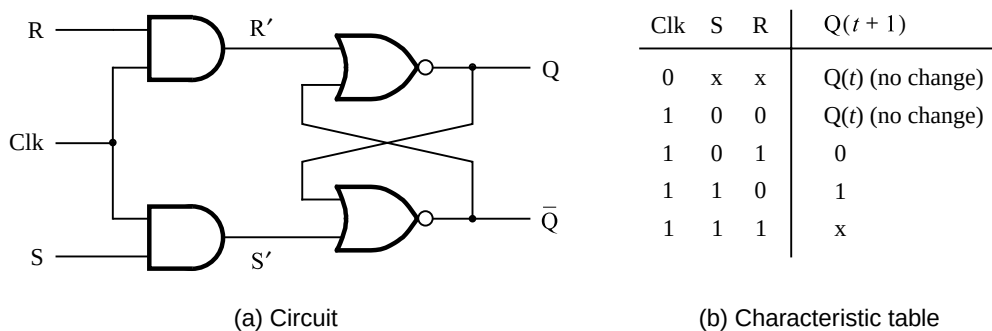
The latch in Figure 7.5a can perform the functions needed for the memory element in Figure 7.1, by connecting the *Set* signal to the S input and *Reset* to the R input. The Q_a output provides the desired *On/Off* signal. To initialize the operation of the alarm system, the latch is reset. Thus the alarm is off. When the sensor generates the logic value 1, the latch is set and Q_a becomes equal to 1. This turns on the alarm mechanism. If the sensor output returns to 0, the latch retains its state where $Q_a = 1$; hence the alarm remains turned on. The only way to turn off the alarm is by resetting the latch, which is accomplished by making the *Reset* input equal to 1.

7.2 GATED SR LATCH

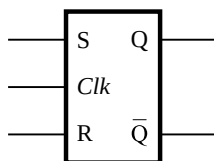
In section 7.1 we saw that the basic SR latch can serve as a useful memory element. It remembers its state when both the S and R inputs are 0. It changes its state in response to changes in the signals on these inputs. The state changes occur at the time when the changes in the signals occur. If we cannot control the time of such changes, then we don't know when the latch may change its state.

In the alarm system of Figure 7.1, it may be desirable to be able to enable or disable the entire system by means of a control input, *Enable*. Thus when enabled, the system would function as described above. In the disabled mode, changing the *Set* input from 0 to 1 would not cause the alarm to turn on. The latch in Figure 7.5a cannot provide the desired operation. But the latch circuit can be modified to respond to the input signals S and R only when *Enable* = 1. Otherwise, it would maintain its state.

The modified circuit is depicted in Figure 7.6a. It includes two AND gates that provide the desired control. When the control signal *Clk* is equal to 0, the S' and R' inputs to the latch will be 0, regardless of the values of signals S and R . Hence the latch will maintain its



(c) Timing diagram



(d) Graphical symbol

Figure 7.6 Gated SR latch.

existing state as long as $Clk = 0$. When Clk changes to 1, the S' and R' signals will be the same as the S and R signals, respectively. Therefore, in this mode the latch will behave as we described in section 7.1. Note that we have used the name Clk for the control signal that allows the latch to be set or reset, rather than call it the *Enable* signal. The reason is that such circuits are often used in digital systems where it is desirable to allow the changes in

the states of memory elements to occur only at well-defined time intervals, as if they were controlled by a clock. The control signal that defines these time intervals is usually called the *clock* signal. The name *Clk* is meant to reflect this nature of the signal.

Circuits of this type, which use a control signal, are called *gated latches*. Because our circuit exhibits set and reset capability, it is called a *gated SR latch*. Figure 7.6b describes its behavior. It defines the state of the Q output at time $t + 1$, namely, $Q(t + 1)$, as a function of the inputs S , R , and Clk . When $Clk = 0$, the latch will remain in the state it is in at time t , that is, $Q(t)$, regardless of the values of inputs S and R . This is indicated by specifying $S = x$ and $R = x$, where x means that the signal value can be either 0 or 1. (Recall that we already used this notation in Chapter 4.) When $Clk = 1$, the circuit behaves as the basic latch in Figure 7.5. It is set by $S = 1$ and reset by $R = 1$. The last row of the table, where $S = R = 1$, shows that the state $Q(t + 1)$ is undefined because we don't know whether it will be 0 or 1. This corresponds to the situation described in section 7.1 in conjunction with the timing diagram in Figure 7.5 at time t_{10} . At this time both S and R inputs go from 1 to 0, which causes the oscillatory behavior that we discussed. If $S = R = 1$, this situation will occur as soon as Clk goes from 1 to 0. To ensure a meaningful operation of the gated SR latch, it is essential to avoid the possibility of having both the S and R inputs equal to 1 when Clk changes from 1 to 0.

A timing diagram for the gated SR latch is given in Figure 7.6c. It shows Clk as a periodic signal that is equal to 1 at regular time intervals to suggest that this is how the clock signal usually appears in a real system. The diagram presents the effect of several combinations of signal values. Observe that we have labeled one output as Q and the other as its complement $\bar{Q}$, rather than Q_a and Q_b as in Figure 7.5. Since the undefined mode, where $S = R = 1$, must be avoided in practice, the normal operation of the latch will have the outputs as complements of each other. Moreover, we will often say that the latch is *set* when $Q = 1$, and it is *reset* when $Q = 0$. A graphical symbol for the gated SR latch is given in Figure 7.6d.

7.2.1 GATED SR LATCH WITH NAND GATES

So far we have implemented the basic latch with cross-coupled NOR gates. We can also construct the latch with NAND gates. Using this approach, we can implement the gated SR latch as depicted in Figure 7.7. The behavior of this circuit is described by the table in Figure 7.6b. Note that in this circuit, the clock is gated by NAND gates, rather than by

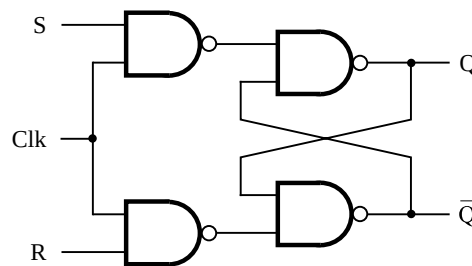


Figure 7.7 Gated SR latch with NAND gates.

AND gates. Note also that the S and R inputs are reversed in comparison with the circuit in Figure 7.6a. The circuit with NAND gates requires fewer transistors than the circuit with AND gates. We will use the circuit in Figure 7.7, in preference to the circuit in Figure 7.6a.

7.3 GATED D LATCH

In section 7.2 we presented the gated SR latch and showed how it can be used as the memory element in the alarm system of Figure 7.1. This latch is useful for many other applications. In this section we describe another gated latch that is even more useful in practice. It has a single data input, called D , and it stores the value on this input, under the control of a clock signal. It is called a *gated D latch*.

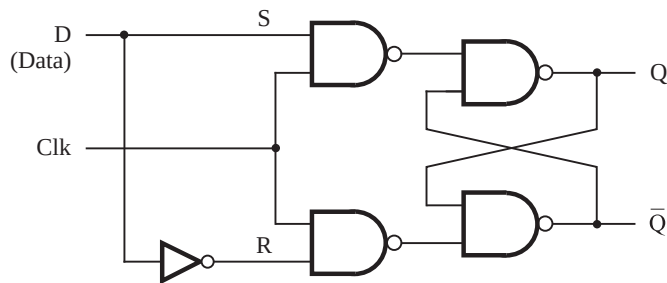
To motivate the need for a gated D latch, consider the adder/subtractor unit discussed in Chapter 5 (Figure 5.13). When we described how that circuit is used to add numbers, we did not discuss what is likely to happen with the sum bits that are produced by the adder. Adder/subtractor units are often used as part of a computer. The result of an addition or subtraction operation is often used as an operand in a subsequent operation. Therefore, it is necessary to be able to remember the values of the sum bits generated by the adder until they are needed again. We might think of using the basic latches to remember these bits, one bit per latch. In this context, instead of saying that a latch remembers the value of a bit, it is more illuminating to say that the latch *stores* the value of the bit or simply “stores the bit.” We should think of the latch as a storage element.

But can we obtain the desired operation using the basic latches? We can certainly reset all latches before the addition operation begins. Then we would expect that by connecting a sum bit to the S input of a latch, the latch would be set to 1 if the sum bit has the value 1; otherwise, the latch would remain in the 0 state. This would work fine if all sum bits are 0 at the start of the addition operation and, after some propagation delay through the adder, some of these bits become equal to 1 to give the desired sum. Unfortunately, the propagation delays that exist in the adder circuit cause a big problem in this arrangement. Suppose that we use a ripple-carry adder. When the X and Y inputs are applied to the adder, the sum outputs may alternate between 0 and 1 a number of times as the carries ripple through the circuit. This situation was illustrated in the timing diagram in Figure 5.21. The problem is that if we connect a sum bit to the S input of a latch, then if the sum bit is temporarily a 1 and then settles to 0 in the final result, the latch will remain set to 1 erroneously.

The problem caused by the alternating values of the sum bits in the adder could be solved by using the gated SR latches, instead of the basic latches. Then we could arrange that the clock signal is 0 during the time needed by the adder to produce a correct sum. After allowing for the maximum propagation delay in the adder circuit, the clock should go to 1 to store the values of the sum bits in the gated latches. As soon as the values have been stored, the clock can return to 0, which ensures that the stored values will be retained until the next time the clock goes to 1. To achieve the desired operation, we would also have to reset all latches to 0 prior to loading the sum-bit values into these latches. This is an awkward way of dealing with the problem, and it is preferable to use the gated D latches instead.

Figure 7.8a shows the circuit for a gated D latch. It is based on the gated SR latch, but instead of using the S and R inputs separately, it has just one data input, D . For convenience we have labeled the points in the circuit that are equivalent to the S and R inputs. If $D = 1$, then $S = 1$ and $R = 0$, which forces the latch into the state $Q = 1$. If $D = 0$, then $S = 0$ and $R = 1$, which causes $Q = 0$. Of course, the changes in state occur only when $Clk = 1$.

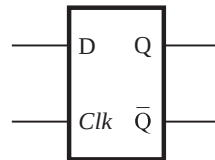
It is important to observe that in this circuit it is impossible to have the troublesome situation where $S = R = 1$. In the gated D latch, the output Q merely tracks the value of the input D while $Clk = 1$. As soon as Clk goes to 0, the state of the latch is frozen until the next time the clock signal goes to 1. Therefore, the gated D latch stores the value of the D



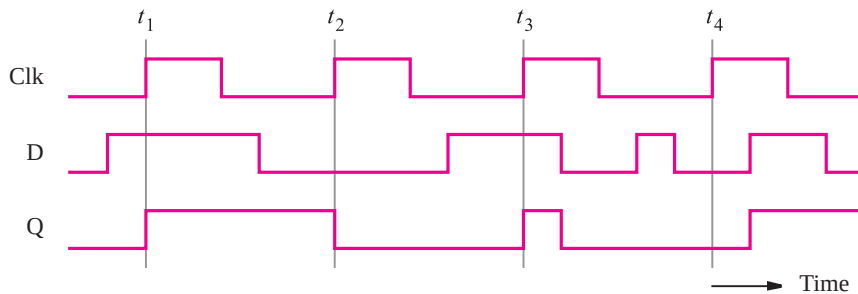
(a) Circuit

Clk	D	$Q(t+1)$
0	x	$Q(t)$
1	0	0
1	1	1

(b) Characteristic table



(c) Graphical symbol



(d) Timing diagram

Figure 7.8 Gated D latch.

input seen at the time the clock changes from 1 to 0. Figure 7.8 also gives the characteristic table, the graphical symbol, and the timing diagram for the gated D latch.

The timing diagram illustrates what happens if the D signal changes while $Clk = 1$. During the third clock pulse, starting at t_3 , the output Q changes to 1 because $D = 1$. But midway through the pulse D goes to 0, which causes Q to go to 0. This value of Q is stored when Clk changes to 0. Now no further change in the state of the latch occurs until the next clock pulse, at t_4 . The key point to observe is that as long as the clock has the value 1, the Q output follows the D input. But when the clock has the value 0, the Q output cannot change. In Chapter 3 we saw that the logic values are implemented as low and high voltage levels. Since the output of the gated D latch is controlled by the level of the clock input, the latch is said to be *level sensitive*. The circuits in Figures 7.6 through 7.8 are level sensitive. We will show in section 7.4 that it is possible to design storage elements for which the output changes only at the point in time when the clock changes from one value to the other. Such circuits are said to be *edge triggered*.

At this point we should reconsider the circuit in Figure 7.3. Careful examination of that circuit shows that it behaves in exactly the same way as the circuit in Figure 7.8a. The *Data* and *Load* inputs correspond to the D and Clk inputs, respectively. The *Output*, which has the same signal value as point A, corresponds to the Q output. Point B corresponds to $\bar{Q}$. Therefore, the circuit in Figure 7.3 is also a gated D latch. An advantage of this circuit is that it can be implemented using fewer transistors than the circuit in Figure 7.8a.

7.3.1 EFFECTS OF PROPAGATION DELAYS

In the previous discussion we ignored the effects of propagation delays. In practical circuits it is essential to take these delays into account. Consider the gated D latch in Figure 7.8a. It stores the value of the D input that is present at the time the clock signal changes from 1 to 0. It operates properly if the D signal is stable (that is, not changing) at the time Clk goes from 1 to 0. But it may lead to unpredictable results if the D signal also changes at this time. Therefore, the designer of a logic circuit that generates the D signal must ensure that this signal is stable when the critical change in the clock signal takes place.

Figure 7.9 illustrates the critical timing region. The minimum time that the D signal must be stable prior to the negative edge of the Clk signal is called the *setup time*, t_{su} , of the

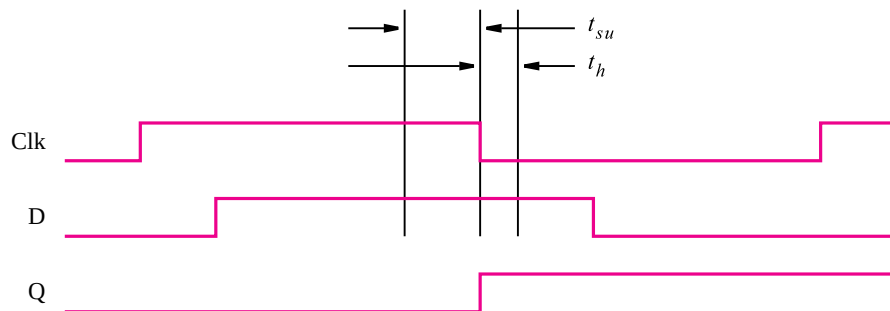


Figure 7.9 Setup and hold times.

latch. The minimum time that the D signal must remain stable after the negative edge of the Clk signal is called the *hold time*, t_h , of the latch. The values of t_{su} and t_h depend on the technology used. Manufacturers of integrated circuit chips provide this information on the data sheets that describe their chips. Typical values for a modern CMOS technology may be $t_{su} = 0.3$ ns and $t_h = 0.2$ ns. We will give examples of how setup and hold times affect the speed of operation of circuits in section 7.13. The behavior of storage elements when setup or hold times are violated is discussed in section 10.3.3.

7.4 MASTER-SLAVE AND EDGE-TRIGGERED D FLIP-FLOPS

In the level-sensitive latches, the state of the latch keeps changing according to the values of input signals during the period when the clock signal is active (equal to 1 in our examples). As we will see in sections 7.8 and 7.9, there is also a need for storage elements that can change their states no more than once during one clock cycle. We will discuss two types of circuits that exhibit such behavior.

7.4.1 MASTER-SLAVE D FLIP-FLOP

Consider the circuit given in Figure 7.10a, which consists of two gated D latches. The first, called *master*, changes its state while $Clock = 1$. The second, called *slave*, changes its state while $Clock = 0$. The operation of the circuit is such that when the clock is high, the master tracks the value of the D input signal and the slave does not change. Thus the value of Q_m follows any changes in D , and the value of Q_s remains constant. When the clock signal changes to 0, the master stage stops following the changes in the D input. At the same time, the slave stage responds to the value of the signal Q_m and changes state accordingly. Since Q_m does not change while $Clock = 0$, the slave stage can undergo at most one change of state during a clock cycle. From the external observer's point of view, namely, the circuit connected to the output of the slave stage, the master-slave circuit changes its state at the negative-going edge of the clock. The *negative edge* is the edge where the clock signal changes from 1 to 0. Regardless of the number of changes in the D input to the master stage during one clock cycle, the observer of the Q_s signal will see only the change that corresponds to the D input at the negative edge of the clock.

The circuit in Figure 7.10 is called a *master-slave D flip-flop*. The term *flip-flop* denotes a storage element that changes its output state at the edge of a controlling clock signal. The timing diagram for this flip-flop is shown in Figure 7.10b. A graphical symbol is given in Figure 7.10c. In the symbol we use the $>$ mark to denote that the flip-flop responds to the “active edge” of the clock. We place a bubble on the clock input to indicate that the active edge for this particular circuit is the negative edge.

7.4.2 EDGE-TRIGGERED D FLIP-FLOP

The output of the master-slave D flip-flop in Figure 7.10a responds on the negative edge of the clock signal. The circuit can be changed to respond to the positive clock edge by connecting the slave stage directly to the clock and the master stage to the complement of

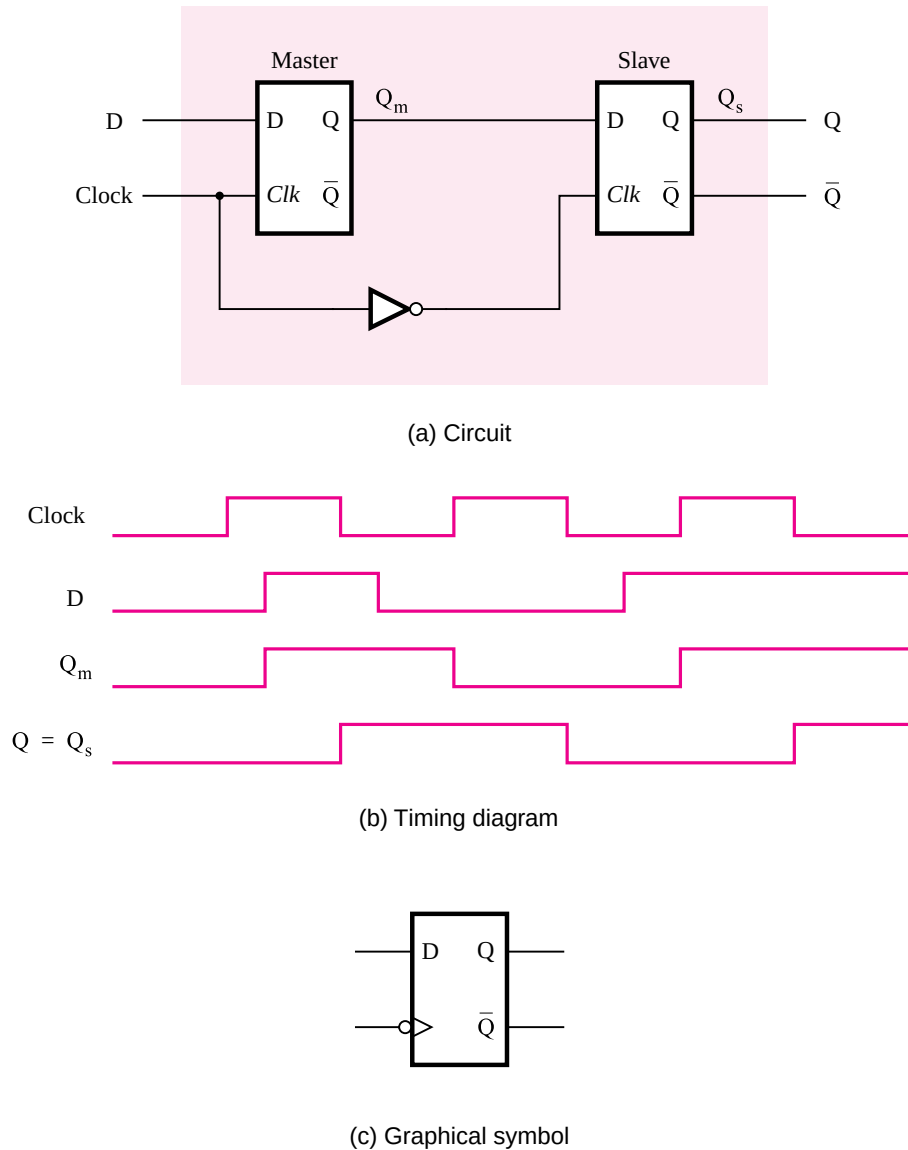
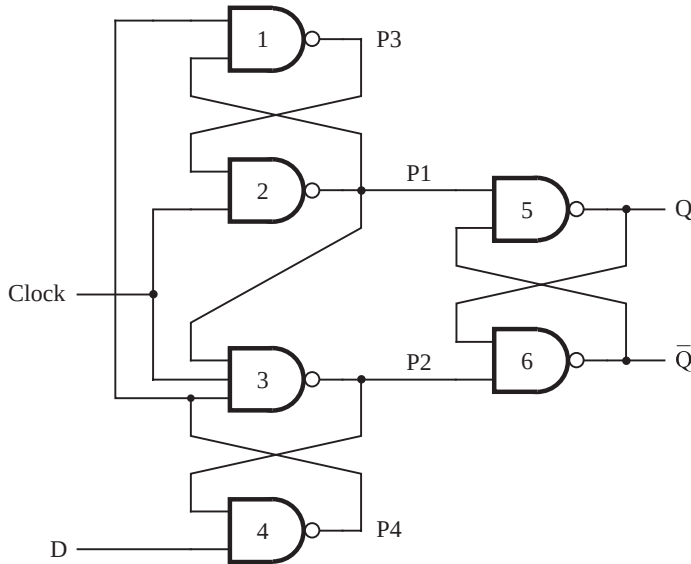
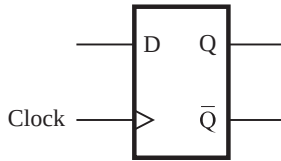


Figure 7.10 Master-slave D flip-flop.

the clock. A different circuit that accomplishes the same task is presented in Figure 7.11a. It requires only six NAND gates and, hence, fewer transistors. The operation of the circuit is as follows. When *Clock* = 0, the outputs of gates 2 and 3 are high. Thus $P1 = P2 = 1$, which maintains the output latch, comprising gates 5 and 6, in its present state. At the same time, the signal *P3* is equal to *D*, and *P4* is equal to its complement $\bar{D}$. When *Clock* changes



(a) Circuit



(b) Graphical symbol

Figure 7.11 A positive-edge-triggered D flip-flop.

to 1, the following changes take place. The values of $P3$ and $P4$ are transmitted through gates 2 and 3 to cause $P1 = \bar{D}$ and $P2 = D$, which sets $Q = D$ and $\bar{Q} = \bar{D}$. To operate reliably, $P3$ and $P4$ must be stable when $Clock$ changes from 0 to 1. Hence the setup time of the flip-flop is equal to the delay from the D input through gates 4 and 1 to $P3$. The hold time is given by the delay through gate 3 because once $P2$ is stable, the changes in D no longer matter.

For proper operation it is necessary to show that, after $Clock$ changes to 1, any further changes in D will not affect the output latch as long as $Clock = 1$. We have to consider two cases. Suppose first that $D = 0$ at the positive edge of the clock. Then $P2 = 0$, which will keep the output of gate 4 equal to 1 as long as $Clock = 1$, regardless of the value of the D input. The second case is if $D = 1$ at the positive edge of the clock. Then $P1 = 0$, which forces the outputs of gates 1 and 3 to be equal to 1, regardless of the D input. Therefore, the flip-flop ignores changes in the D input while $Clock = 1$.

Figure 7.11*b* gives a graphical symbol for this flip-flop. The clock input indicates that the positive edge of the clock is the active edge. A similar circuit, constructed with NOR gates, can be used as a negative-edge-triggered flip-flop.

Level-Sensitive versus Edge-Triggered Storage Elements

Figure 7.12 shows three different types of storage elements that are driven by the same data and clock inputs. The first element is a gated D latch, which is level sensitive. The second one is a positive-edge-triggered D flip-flop, and the third one is a negative-edge-triggered D flip-flop. To accentuate the differences between these storage elements, the

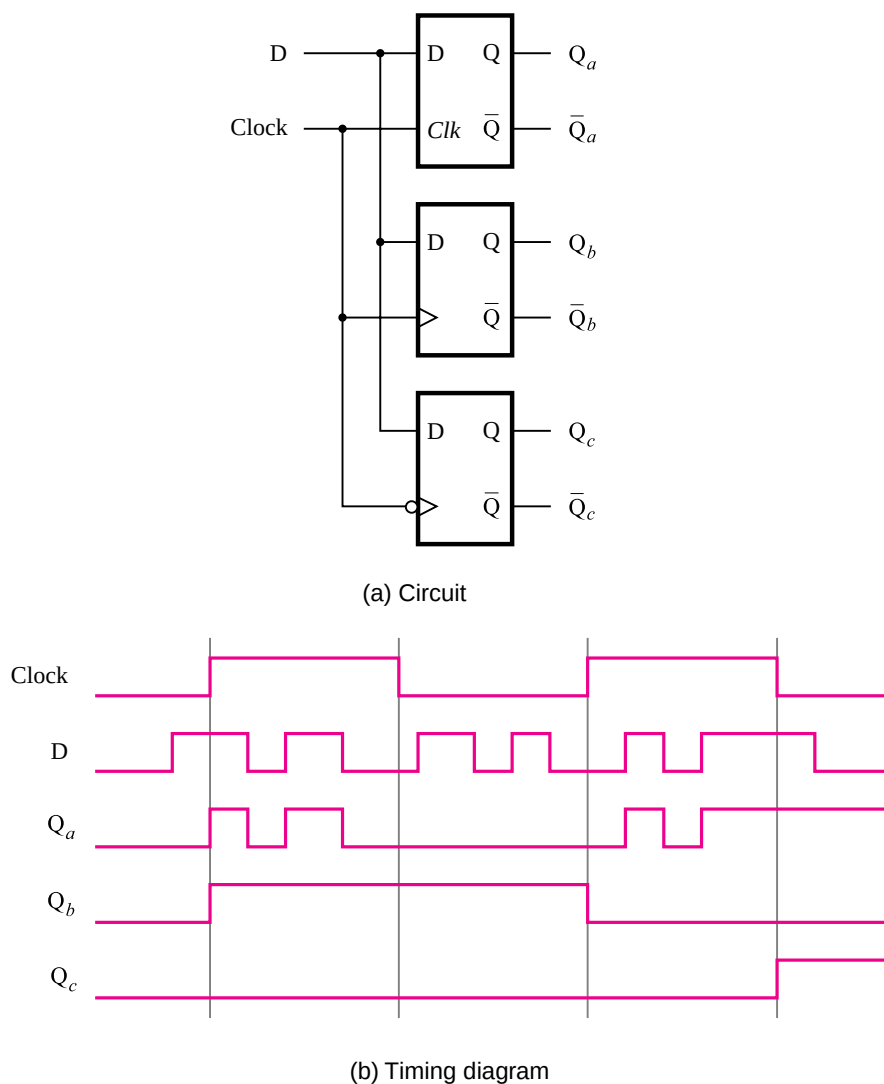


Figure 7.12 Comparison of level-sensitive and edge-triggered D storage elements.

D input changes its values more than once during each half of the clock cycle. Observe that the gated D latch follows the D input as long as the clock is high. The positive-edge-triggered flip-flop responds only to the value of D when the clock changes from 0 to 1. The negative-edge-triggered flip-flop responds only to the value of D when the clock changes from 1 to 0.

7.4.3 D FLIP-FLOPS WITH CLEAR AND PRESET

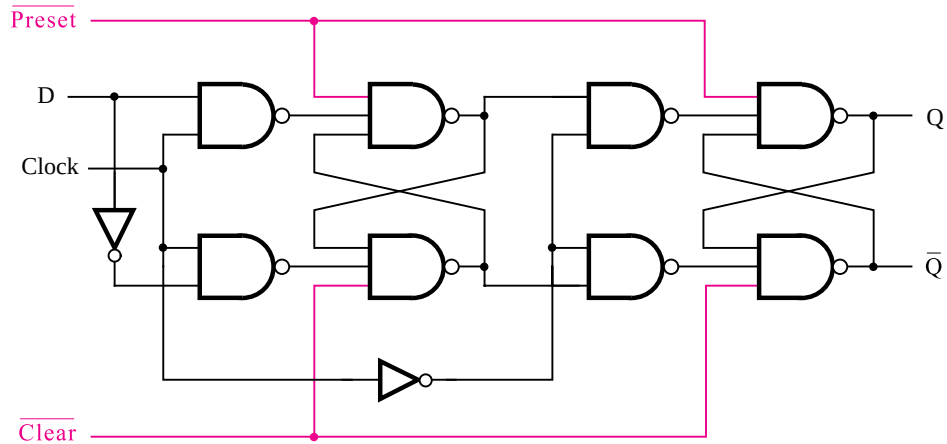
Flip-flops are often used for implementation of circuits that can have many possible states, where the response of the circuit depends not only on the present values of the circuit's inputs but also on the particular state that the circuit is in at that time. We will discuss a general form of such circuits in Chapter 8. A simple example is a counter circuit that counts the number of occurrences of some event, perhaps passage of time. We will discuss counters in detail in section 7.9. A counter comprises a number of flip-flops, whose outputs are interpreted as a number. The counter circuit has to be able to increment or decrement the number. It is also important to be able to force the counter into a known initial state (count). Obviously, it must be possible to clear the count to zero, which means that all flip-flops must have $Q = 0$. It is equally useful to be able to preset each flip-flop to $Q = 1$, to insert some specific count as the initial value in the counter. These features can be incorporated into the circuits of Figures 7.10 and 7.11 as follows.

Figure 7.13a shows an implementation of the circuit in Figure 7.10a using NAND gates. The master stage is just the gated D latch of Figure 7.8a. Instead of using another latch of the same type for the slave stage, we can use the slightly simpler gated SR latch of Figure 7.7. This eliminates one NOT gate from the circuit.

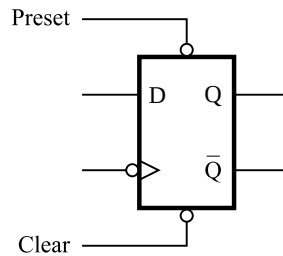
A simple way of providing the clear and preset capability is to add an extra input to each NAND gate in the cross-coupled latches, as indicated in blue. Placing a 0 on the *Clear* input will force the flip-flop into the state $Q = 0$. If *Clear* = 1, then this input will have no effect on the NAND gates. Similarly, *Preset* = 0 forces the flip-flop into the state $Q = 1$, while *Preset* = 1 has no effect. To denote that the *Clear* and *Preset* inputs are active when their value is 0, we placed an overbar on the names in the figure. We should note that the circuit that uses this flip-flop should not try to force both *Clear* and *Preset* to 0 at the same time. A graphical symbol for this flip-flop is shown in Figure 7.13b.

A similar modification can be done on the edge-triggered flip-flop of Figure 7.11a, as indicated in Figure 7.14a. Again, both *Clear* and *Preset* inputs are active low. They do not disturb the flip-flop when they are equal to 1.

In the circuits in Figures 7.13a and 7.14a, the effect of a low signal on either the *Clear* or *Preset* input is immediate. For example, if *Clear* = 0 then the flip-flop goes into the state $Q = 0$ immediately, regardless of the value of the clock signal. In such a circuit, where the *Clear* signal is used to clear a flip-flop without regard to the clock signal, we say that the flip-flop has an *asynchronous clear*. In practice, it is often preferable to clear the flip-flops on the active edge of the clock. Such *synchronous clear* can be accomplished as shown in Figure 7.14c. The flip-flop operates normally when the *Clear* input is equal to 1. But if *Clear* goes to 0, then on the next positive edge of the clock the flip-flop will be cleared to 0. We will examine the clearing of flip-flops in more detail in section 7.10.



(a) Circuit



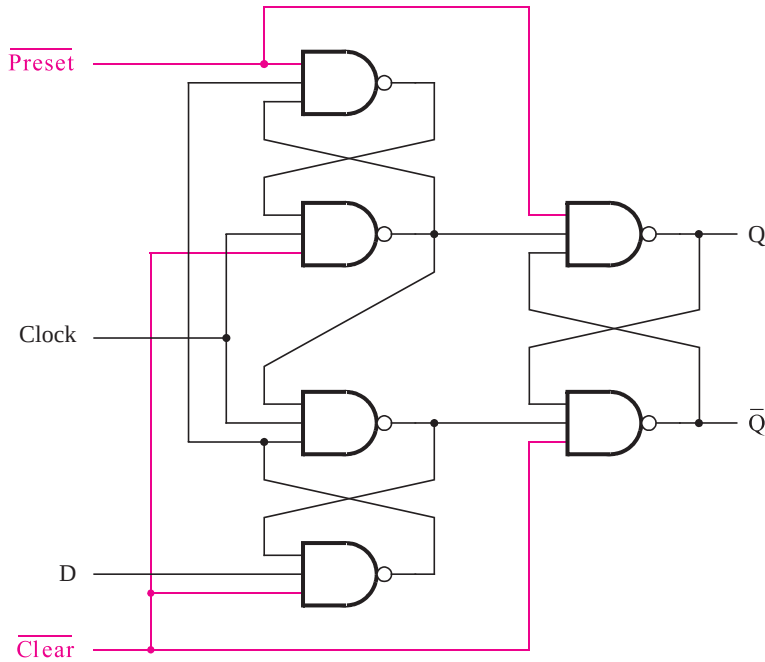
(b) Graphical symbol

Figure 7.13 Master-slave D flip-flop with *Clear* and *Preset*.

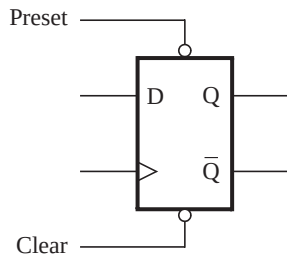
7.4.4 FLIP-FLOP TIMING PARAMETERS

In section 7.3.1 we discussed timing issues related to latch circuits. In practice such issues are equally important for circuits with flip-flops. Figure 7.15a shows a positive-edge triggered flip-flop with asynchronous clear, and part *b* of the figure illustrates some important timing parameters for this flip-flop. Data is loaded into the D input of the flip-flop on a positive clock edge, and this logic value must be stable during the setup time, t_{su} , before the clock edge occurs. The data must remain stable during the hold time, t_h , after the edge. If the setup or hold requirements are not adhered to in a circuit that uses this flip-flop, then it may enter an unstable condition known as *metastability*; we discuss this concept in section 10.3.

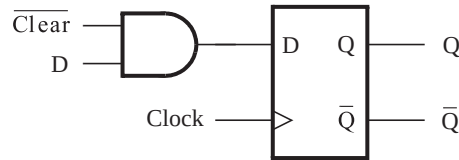
As indicated in Figure 7.15, a clock-to-Q propagation delay, t_{cQ} , is incurred before the value of Q changes after a positive clock edge. In general, the delay may not be



(a) Circuit



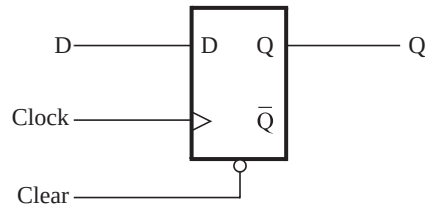
(b) Graphical symbol



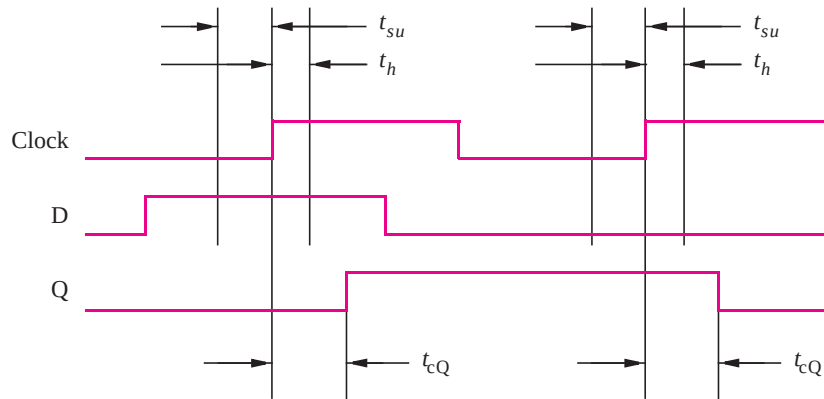
(c) Adding a synchronous clear

Figure 7.14 Positive-edge-triggered D flip-flop with *Clear* and *Preset*.

exactly the same for the cases when Q changes from 1 to 0 or 0 to 1, but we assume for simplicity that these delays are equal. For the flip-flops in a commercial chip, two values are usually specified for t_{cQ} , representing the maximum and minimum delays that may occur in practice. Specifying a range of values when estimating the delays in a chip is a common practice due to many sources of variation in delay that are caused by the chip manufacturing



(a) D flip-flop with asynchronous clear



(b) Timing diagram

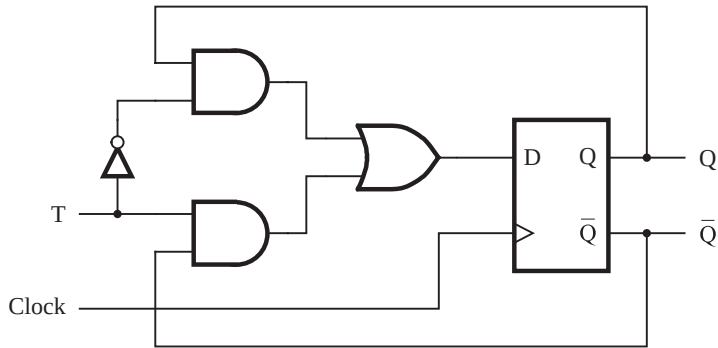
Figure 7.15 Flip-flop timing parameters.

process. In section 7.15 we provide some examples that illustrate the effects of flip-flop timing parameters on the operation of circuits.

7.5 T FLIP-FLOP

The D flip-flop is a versatile storage element that can be used for many purposes. By including some simple logic circuitry to drive its input, the D flip-flop may appear to be a different type of storage element. An interesting modification is presented in Figure 7.16a. This circuit uses a positive-edge-triggered D flip-flop. The *feedback* connections make the input signal D equal to either the value of Q or $\bar{Q}$ under the control of the signal that is labeled T . On each positive edge of the clock, the flip-flop may change its state $Q(t)$. If $T = 0$, then $D = Q$ and the state will remain the same, that is, $Q(t + 1) = Q(t)$. But if $T = 1$, then $D = \bar{Q}$ and the new state will be $Q(t + 1) = \bar{Q}(t)$. Therefore, the overall operation of the circuit is that it retains its present state if $T = 0$, and it reverses its present state if $T = 1$.

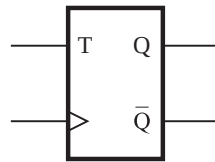
The operation of the circuit is specified in the form of a characteristic table in Figure 7.16b. Any circuit that implements this table is called a *T flip-flop*. The name T flip-flop



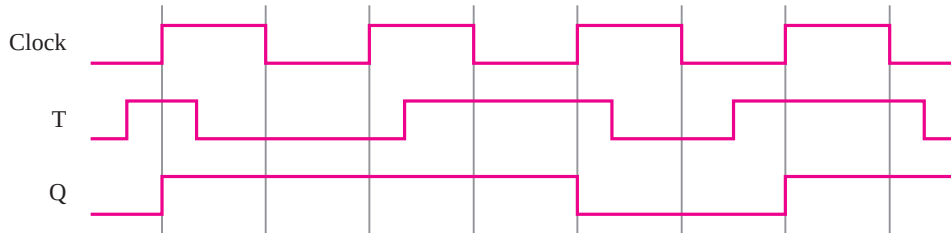
(a) Circuit

T	$Q(t+1)$
0	$Q(t)$
1	$\bar{Q}(t)$

(b) Characteristic table



(c) Graphical symbol



(d) Timing diagram

Figure 7.16 T flip-flop.

derives from the behavior of the circuit, which “toggles” its state when $T = 1$. The toggle feature makes the T flip-flop a useful element for building counter circuits, as we will see in section 7.9.

7.5.1 CONFIGURABLE FLIP-FLOPS

For some circuits one type of flip-flop may lead to a more efficient implementation than a different type of flip-flop. In general purpose chips like PLDs, the flip-flops that are provided are sometimes *configurable*, which means that a flip-flop circuit can be configured to be

either D, T, or some other type. For example, in some PLDs the flip-flops can be configured as either D or T types (see problems 7.6 and 7.8).

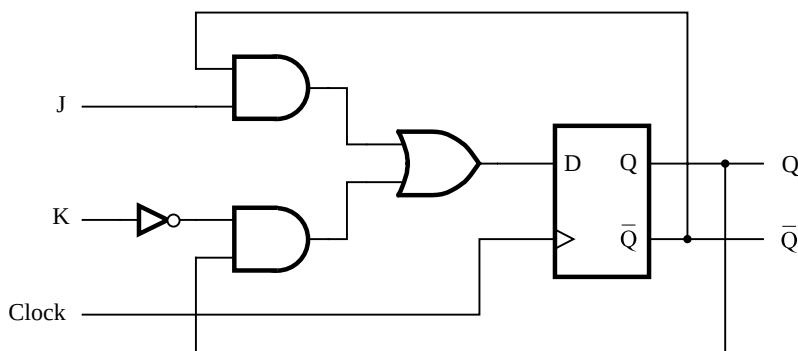
7.6 JK FLIP-FLOP

Another interesting circuit can be derived from Figure 7.16a. Instead of using a single control input, T , we can use two inputs, J and K , as indicated in Figure 7.17a. For this circuit the input D is defined as

$$D = J\bar{Q} + \bar{K}Q$$

A corresponding characteristic table is given in Figure 7.17b. The circuit is called a *JK flip-flop*. It combines the behaviors of SR and T flip-flops in a useful way. It behaves as the SR flip-flop, where $J = S$ and $K = R$, for all input values except $J = K = 1$. For the latter case, which has to be avoided in the SR flip-flop, the JK flip-flop toggles its state like the T flip-flop.

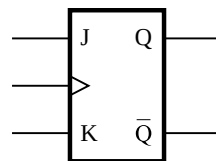
The JK flip-flop is a versatile circuit. It can be used for straight storage purposes, just like the D and SR flip-flops. But it can also serve as a T flip-flop by connecting the J and K inputs together.



(a) Circuit

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\bar{Q}(t)$

(b) Characteristic table



(c) Graphical symbol

Figure 7.17 JK flip-flop.

7.7 SUMMARY OF TERMINOLOGY

We have used the terminology that is quite common. But the reader should be aware that different interpretations of the terms *latch* and *flip-flop* can be found in the literature. Our terminology can be summarized as follows:

Basic latch is a feedback connection of two NOR gates or two NAND gates, which can store one bit of information. It can be set to 1 using the *S* input and reset to 0 using the *R* input.

Gated latch is a basic latch that includes input gating and a control input signal. The latch retains its existing state when the control input is equal to 0. Its state may be changed when the control signal is equal to 1. In our discussion we referred to the control input as the clock. We considered two types of gated latches:

- **Gated SR latch** uses the *S* and *R* inputs to set the latch to 1 or reset it to 0, respectively.
- **Gated D latch** uses the *D* input to force the latch into a state that has the same logic value as the *D* input.

A **flip-flop** is a storage element based on the gated latch principle, which can have its output state changed only on the edge of the controlling clock signal. We considered two types:

- **Edge-triggered flip-flop** is affected only by the input values present when the active edge of the clock occurs.
- **Master-slave flip-flop** is built with two gated latches. The master stage is active during half of the clock cycle, and the slave stage is active during the other half. The output value of the flip-flop changes on the edge of the clock that activates the transfer into the slave stage.